Red Pitaya Signal Generator + Oscilloscope GUI for Data Acquisition and Analysis

Christopher Lock

Table of Contents

Introduction

- Language + Tools
- Useful Links

Setup

- Connect to SCPI server
- Calibration
- File Hierarchy

GUI

- Signal Generator
- Data Acquisition
- Live Data Acquisition
- Decimation Factor
- Deep Memory Acquisition
- Reserving Memory for DMA
- File Readers
- In-depth Guide

Introduction

This project is a Python based application designed to control the Red Pitaya using a custom GUI. The program connects to the Red Pitaya SCPI server, letting users to generate signals, capture waveforms, and perform deep memory acquisition (DMA). Data can be saved to CSV and NPZ files within the interface. The GUI is built with PyQt5 and uses Pyqtgraph for fast plotting, offering an easy tool to use for measurement and analysis.

Language + Tools

- Python 3
- PyQt5 for GUI
- Pyqtgraph for fast live plotting
- Red Pitaya Scpi Server for remote communication: https://redpitaya.readthedocs.io/en/latest/appsFeatures/remoteControl/scpi.html

Useful Links

Github:

https://github.com/chrloc313/red-pitaya-custom-oscilloscope-interface-data-acquisition-tool

Red Pitaya general documentation: https://redpitaya.readthedocs.io/en/latest/intro.html

How to establish connection to Red Pitaya:
https://redpitaya.readthedocs.io/en/latest/quickStart/first.html

Setup

Required python modules: PyQt5, pyqtgraph

Required files on Github: main.py, main_window.py, acq_plot.py, frame_averager.py, ui_components.py, save_data.py, redpitaya_scpi_core.py

Github: https://github.com/chrloc313/red-pitaya-custom-oscilloscope-interface-data-acquisition-tool

redpitaya_scpi_core.py was taken from: https://github.com/RedPitaya/RedPitaya-Examples/blob/dev/python/lib/redpitaya_scpi_core.py

Connect to SCPI server

Follow the red pitaya SCPI server guide to setup a remote connection between your laptop and computer: https://redpitaya.readthedocs.io/en/latest/appsFeatures/remoteControl/scpi.html

In main.py, modify the IP address to the address of your Red Pitaya board by modifying the code. Host IP should be in the form of a string.

Examples: ‘rp-xxxxxx.local’ or ‘192.168.9.10’

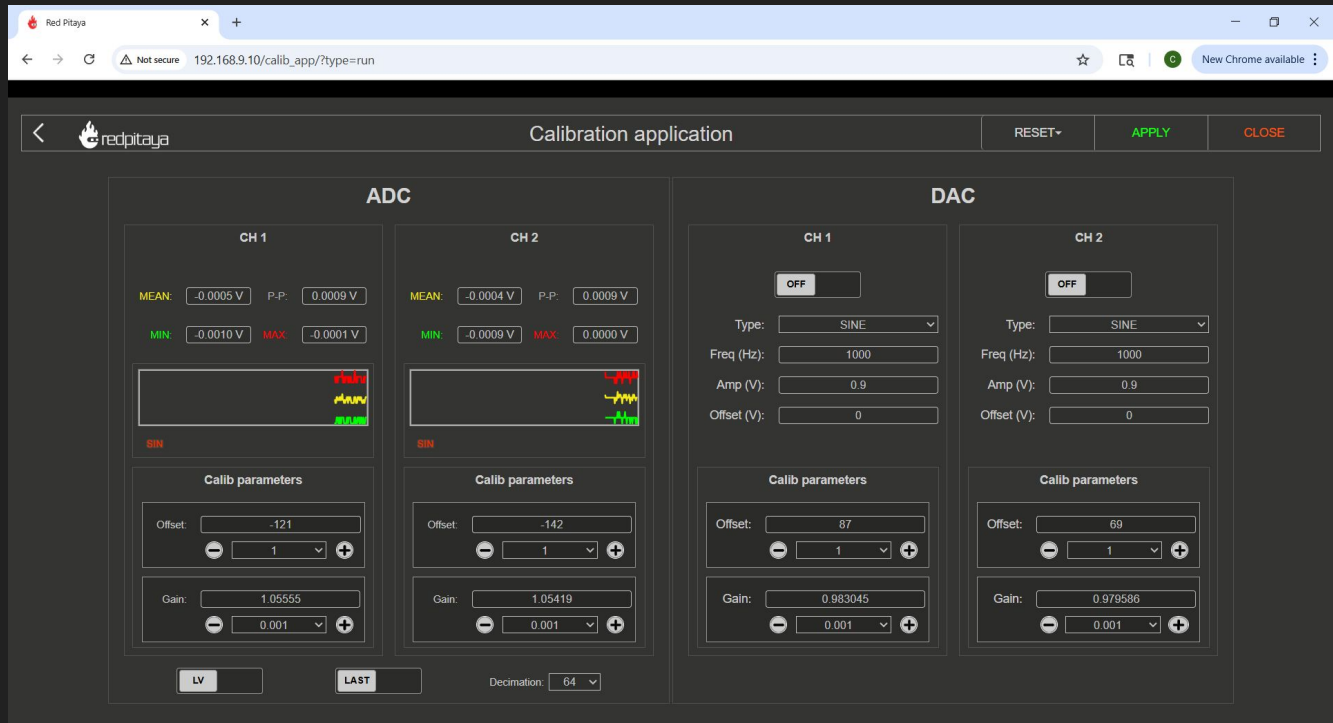
Here is more: https://redpitaya.readthedocs.io/en/latest/quickStart/first.html

```
IP = 'rp-f06d1a.local'
rp = scpi.scpi(IP)
rp_setup(rp)
```

Calibration

To calibrate go to: Redpitaya Applications → System → Calibration → Manual DC

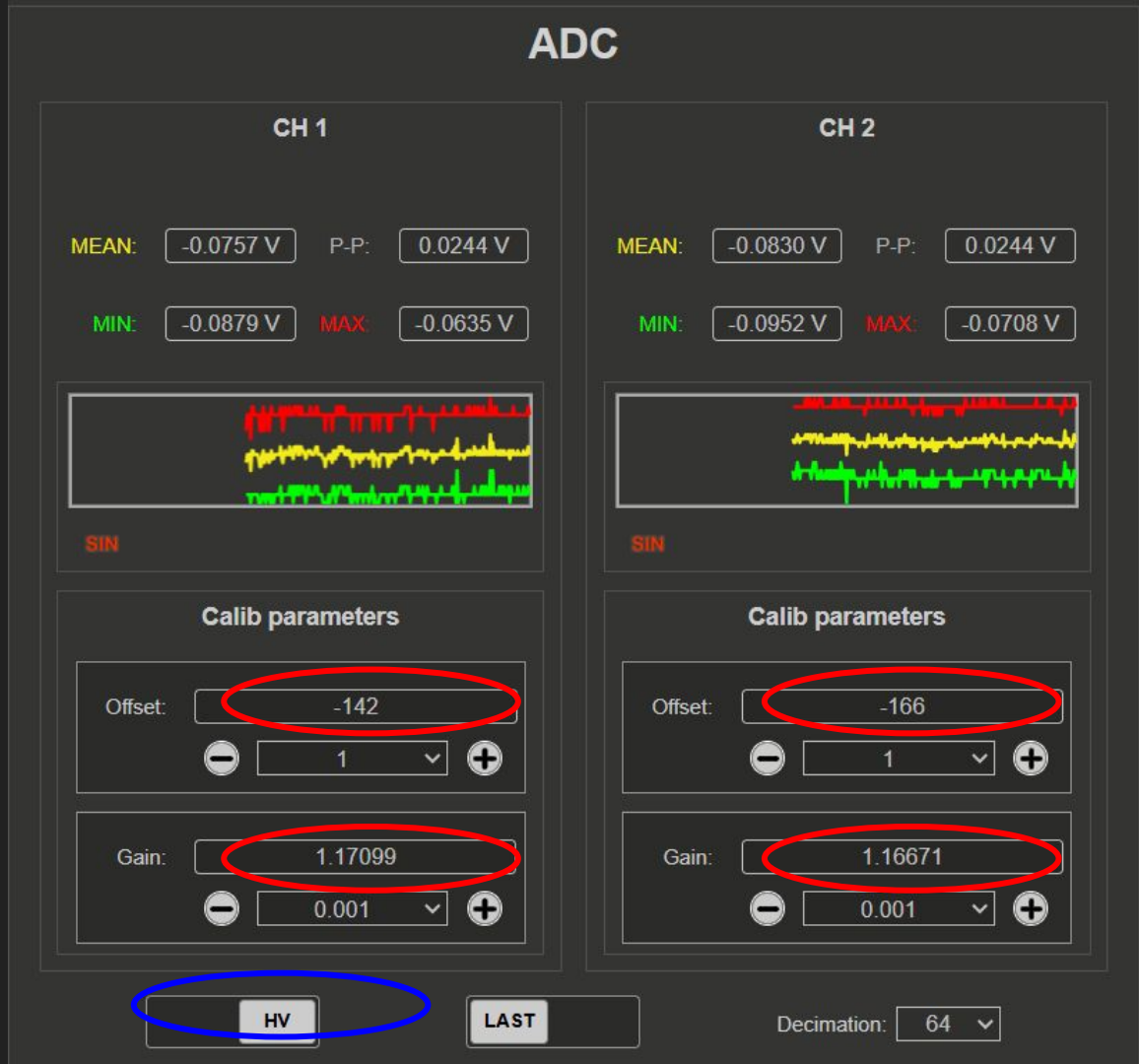
```
19  class MainWindow(QMainWindow):
20
21      MAX_AVG_FRAMES = 100
22      MAX_SAVE_FRAMES = 100
23      FRAME_SIZE = 16384
24
25      OFFSET_1 = -121
26      GAIN_1 = 1.05555
27      OFFSET_2 = -142
28      GAIN_2 = 1.05419
29
30      SCALE_FACTOR = 20.0 / 8192
31
```



Calibration

Switch to HV and modify these constants in the main_window.py file to match the circled parameters of your Red Pitaya:

```
OFFSET_1 = -142
GAIN_1 = 1.17099
OFFSET_2 = -166
GAIN_2 = 1.16671
```



File Hierarchy:

main.py

| – redpitaya_scpi_core.py

main_window.py _____

| | |

acq_plot.py frame_averager.py ui_components.py

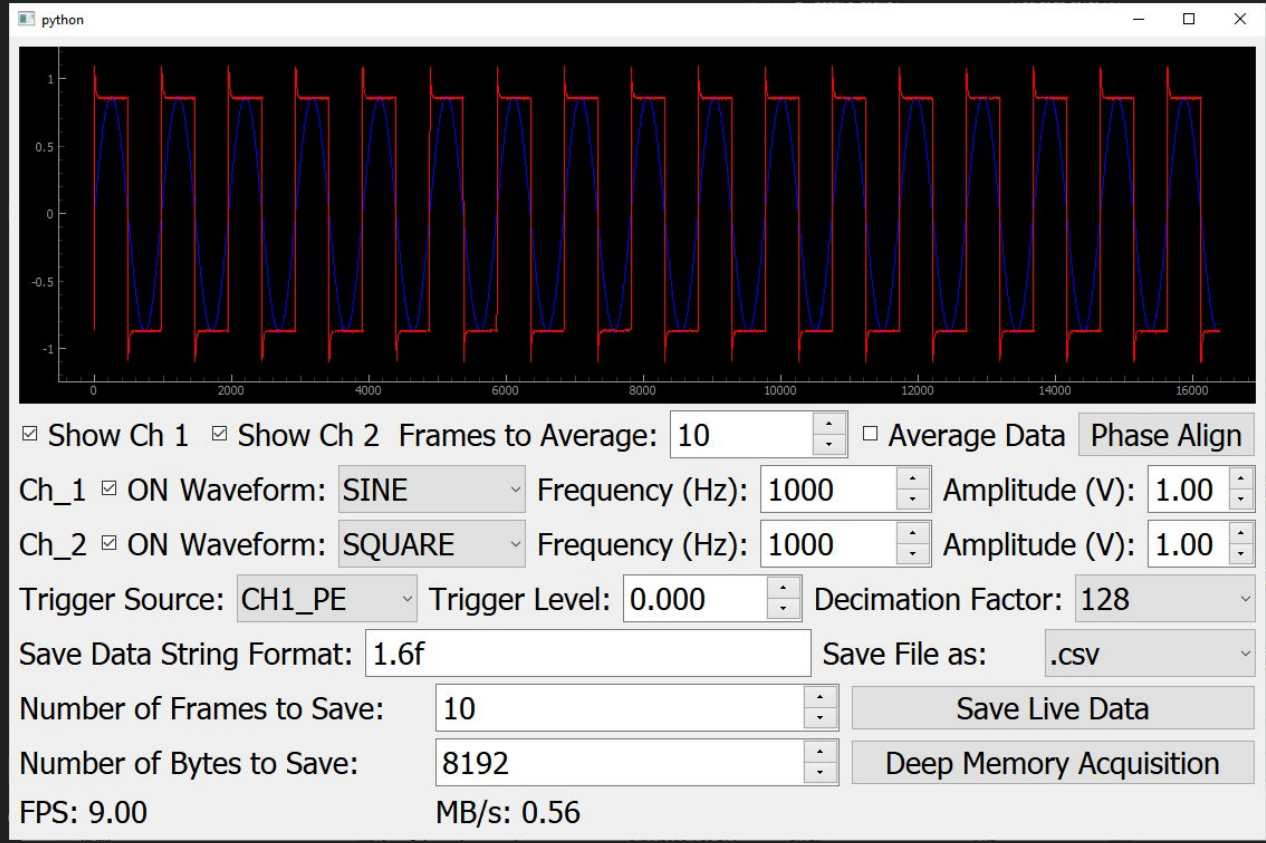
|

save_data.py

GUI

Run main.py

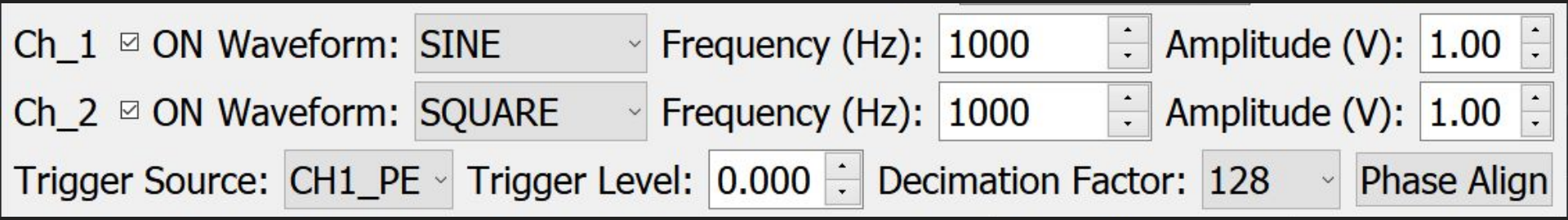
This is what the GUI should look like if the program executes successfully.



Signal Generator

Signal generator is mainly a tool used for testing.

- Generates 2 waveforms from both DACs
  - Can be turned ON and OFF
  - Can change shape, frequency, amplitude
- Trigger Source can be changed to external positive/negative edge



Data Acquisition

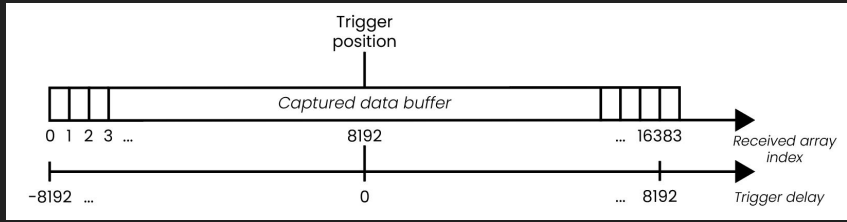
To acquire data, ‘ACQ:START’ and‘ ACQ:TRIG:Condition’ must be run every time in that order

After the trigger condition is met, each acquisition transfers 16384 samples per channel

A circular memory buffer writes data to a data buffer

‘ACQ:TRIG:DLY 8192’ is run to get

16384 samples after the trigger position



Source of images and more details:
https://redpitaya.readthedocs.io/en/latest/appsFeatures/introAcqGen.html

Live Data Acquisition

The live data is obtained by acquiring data over and over.

From testing at 1 kHz, the program hovers around 9-12 frames/acquisitions per second at around 0.5 - 0.8 MiB per second.

The live graph is not truly continuous due to limitation speeds for receiving data from SCPI.



Decimation factor

Base Sampling Rate = 125 MSa/s

Decimation: reduce samples by keeping every Nth sample

Decimation Factor (N)	Sampling Rate = (125 / Decimation Factor) MSa/s
1	125 MSa/s
8	15.625 MSa/s
64	1.953125 MSa/s
128	0.9765625 MSa/s
256	0.48828125 MSa/s
512	0.244140625 MSa/s

Deep Memory Acquisition

Because data will be missed in the live graph, using Deep Memory Acquisition is a single acquisition with a very large amount of samples,

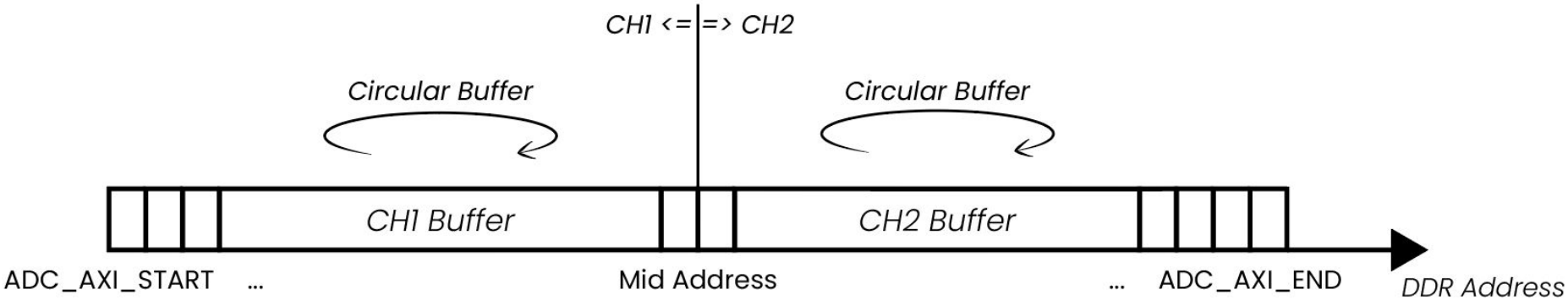
It is used to capture data over a period of time and stored in the DDR, then the data is transferred once data is captured.

https://redpitaya.readthedocs.io/en/latest/appsFeatures/remoteControl/command_list/commands-dmm.html#commands-dmm

Deep Memory Acquisition (DMA)

Works similar to Data Acquisition, but the circular buffer writes the data to reserved memory in DDR, acting as one very large data acquisition.

The data is transferred in chunks using pointers for each channel.



DMA - Reserving Memory

Default RAM memory region is 32 MB

STEMlab 125-14

- 512 MB of RAM
- 100 MB of DDR is reserved for Linux OS, leaving a max 412 MB to store acquired data



DMA - Reserving Memory

The amount of reserved memory must be a multiple of 8192.

- 4096 to use 1 channel, 8192 to use 2 channels
- the closest number divisible by 8192 under 412,000,000 is 411,992,064
- 411,992,064 → Hex: 0x188e8000

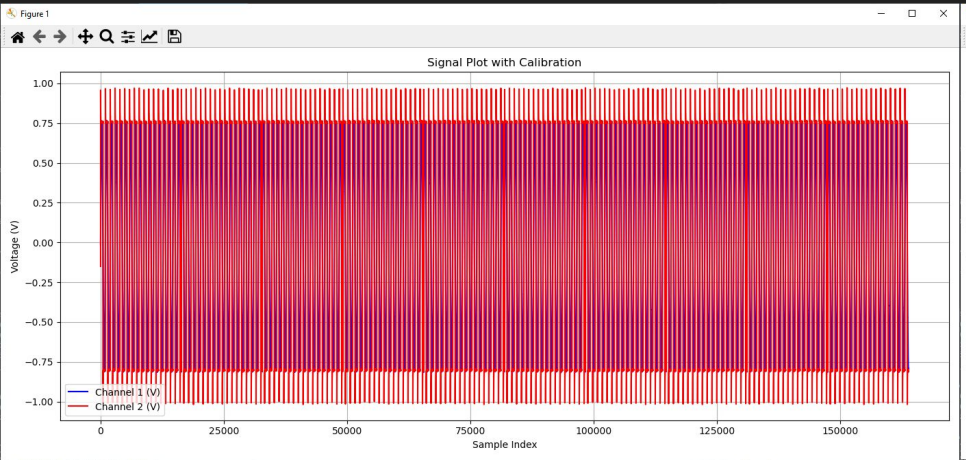
Changing Reserved Memory Guide:
https://redpitaya.readthedocs.io/en/latest/appsFeatures/remoteControl/deepMemoryMode.html#changing-reserved-memory

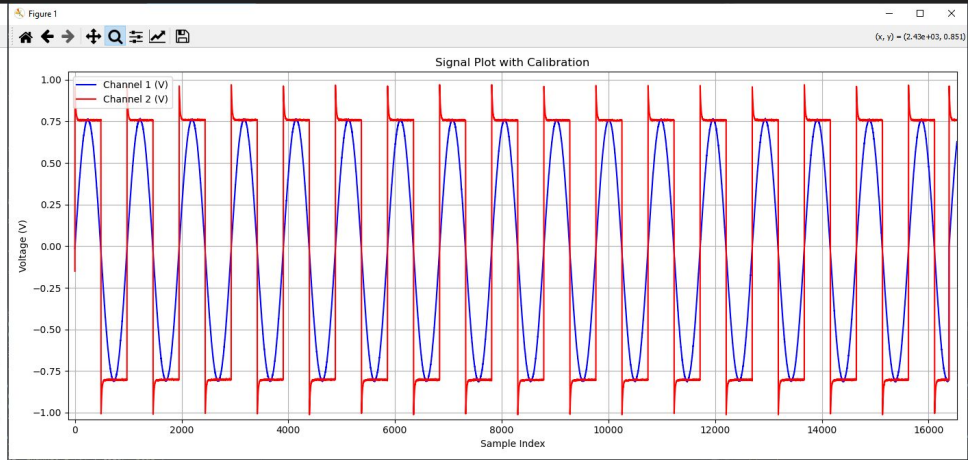
File Readers

File readers only used for .npz files (uses matplotlib to graph)

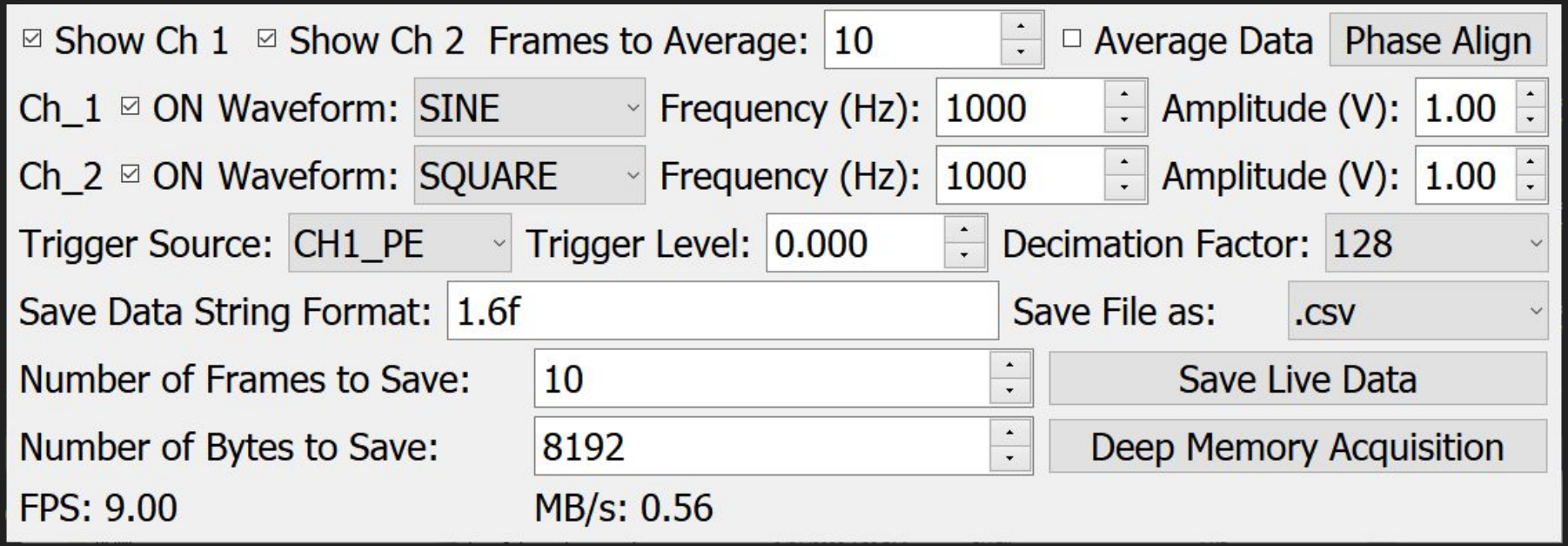
- file_reader_npz_live.py
- file_reader_npz_dma.py

CSV files are too large to be opened in Excel if number of samples per channel exceeds 60





GUI in-depth guide



GUI in-depth guide



Show Ch 1/ Ch 2:

- Shows/hides the data for respective channel if checked/unchecked

Frames to Average:

- Each frame/acquisition is 16384 samples and a running average is kept for the last N frames
- Takes values 1 to 100

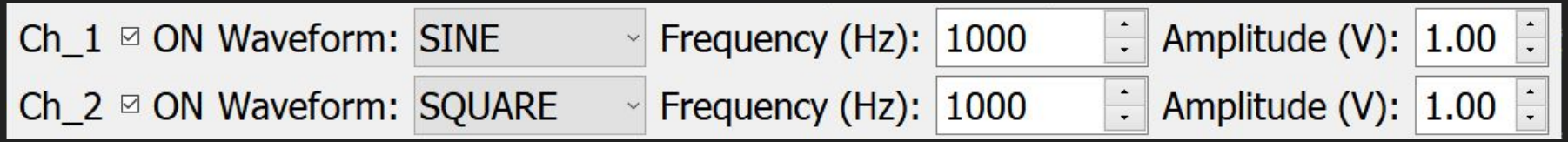
Average Data:

- Starts/stops averaging the last N frames and plots data if checked/unchecked

Phase Align:

- Realigns signal generator
- Misalignment occurs when changing signal frequencies

GUI in-depth guide



ON:

- Turns signal generator for respective channel on/off if checked/unchecked

Waveform:

- Sets the type of wave to be generated
- Options: "SINE", "SQUARE", "TRIANGLE", "SAWU", "SAWD", "PWM", "ARBITRARY", "DC", "DC_NEG"

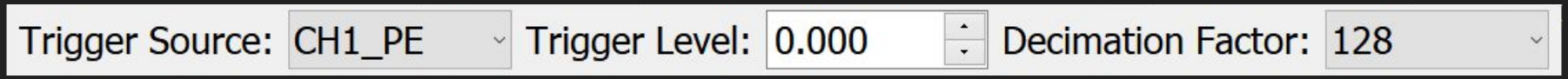
Frequency:

- Sets frequency of wave in Hertz
- Takes values 0 to 62,500,000

Amplitude:

- Sets amplitude of wave in Volts
- Takes values -1 to 1

GUI in-depth guide



Trigger Source:

- Sets the trigger condition for the data acquisition
- Options: "CH1_PE", "CH1_NE", "CH2_PE", "CH2_NE", "EXT_PE", "EXT_NE"
- External positive/negative edge have more practical uses

Trigger Level:

- Sets the trigger level in volts
- Takes values -20 to 20

Decimation Factor:

- Sets the decimation factor → determines acquisition rate
- Options: 1,2,4,8,16,32,64,128,256,512,1024,2048,4096,8192,16384,32768,65536

GUI in-depth guide



Save Data String Format:

- Takes a valid Python format specifier
- For 1.6f, values are stored in fixed-point notation with 6 decimals (e.g., 3.14 → 3.140000)
- Affects only CSV file output

Save File as:

- Can choose between .csv and .npz file to save to
- Does not start saving any data, just sets it to the type for later use

GUI in-depth guide



Number of Frames to Save:

- Takes values between 1 to 100
- Sets the number of frames/acquisition to save

Save Live Data:

- Starts recording data of the last N frames/acquisition in a buffer and saves it to a file of the user's choice
- Can record averages

Number of Bytes to Save:

- Takes values between 8192 and the max reserved memory size
- This number must also be divisible by 8192

Deep Memory Acquisition:

- Starts DMA, using the number of bytes to save, then saves it to a file of the user's choice
- Does not record any averages

Improvements to add

Add feature where the unused 2 bits represent the external trigger

Improve status bar (currently only used when saving)

Add time scale to file readers, currently only decimation rate is available, which time between each data point can be calculated by Decimation Factor/Sampling Rate

Add options to choose between ch1, ch2, or both for DMA (currently locked to both)

Implementing a python API method instead of the SCPI server may give better results for transferring continuous acquisition data, should also have faster DMA transfer speeds